Check Whether Two Strings Are Anagrams

Problem Statement

Given two non-empty strings `s1` and `s2`, consisting only of lowercase English letters, determine whether they are anagrams of each other.

Two strings are anagrams if they contain the same characters with exactly the same frequencies, regardless of their order.

Examples

Example 1

Input:

```
s1 = "geeks"
s2 = "kseeg"
```

Output:

```
true
```

Explanation:
Both strings contain the same characters with the same frequencies.

Example 2

Input:

```
s1 = "allergy"
s2 = "allergyy"
```

Output:

```
false
```

Explanation:
`s2` contains an extra `y`, so the character frequencies are different.

Example 3

Input:

```
s1 = "listen"
s2 = "lists"
```

Output:

```
false
```

Explanation:
The two strings have different lengths and do not contain the same characters.

Approach

Since the strings contain only lowercase English letters, we can use a frequency array of size `26`.

Steps

1. If the lengths of the strings are different, return `false`.
2. Create an integer array of size `26`.
3. For every character in `s1`, increment its frequency.
4. For every character in `s2`, decrement its frequency.
5. If every frequency becomes `0`, the strings are anagrams.
6. Otherwise, they are not anagrams.

Character Mapping

```
'a' - 'a' = 0
'b' - 'a' = 1
'c' - 'a' = 2
...
'z' - 'a' = 25
```

Therefore, each lowercase letter can be stored in the range `0` to `25`.

Code

```java
class Solution {
    public static boolean areAnagrams(String s1, String s2) {
```

```
        if (s1.length() != s2.length()) {
            return false;
        }

        int[] freq = new int[26];

        for (int i = 0; i < s1.length(); i++) {
            freq[s1.charAt(i) - 'a']++;
            freq[s2.charAt(i) - 'a']--;
        }

        for (int i = 0; i < 26; i++) {
            if (freq[i] != 0) {
                return false;
            }
        }

        return true;
    }
}
```

Dry Run

For:

```
s1 = "geeks"
s2 = "kseeg"
```

After processing both strings:

```
g → 0
e → 0
k → 0
s → 0
```

All frequencies are `0`, so:

```
Output: true
```

Complexity Analysis

Let `n` be the length of the strings.

- **Time Complexity:** `O(n)`
- **Space Complexity:** `O(1)`

The frequency array always contains only `26` elements, so its size does not depend on the input size.

Alternative Approach

Another solution is to convert both strings into character arrays and sort them.

```
Time:  O(n log n)
Space: O(n)
```

The frequency-array approach is preferred here because the input contains only lowercase English letters.

Key Interview Takeaway

> When the input is restricted to a fixed character set, such as lowercase English letters, a frequency array can often replace sorting or a HashMap and achieve `O(n)` time with `O(1)` extra space.